

2024

面向对象程序设计

第一讲：面向对象程序设计概述

李际军 lijijun@cs.zju.edu.cn



■ 课时：

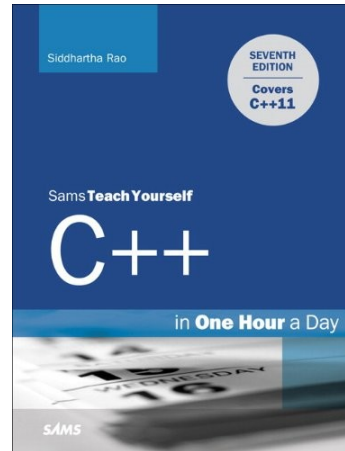
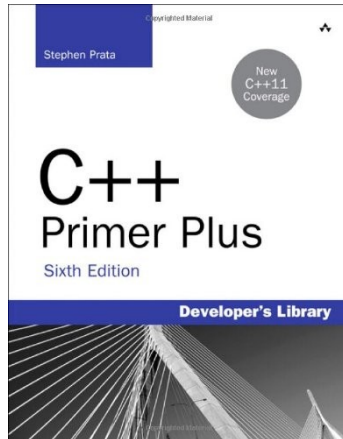
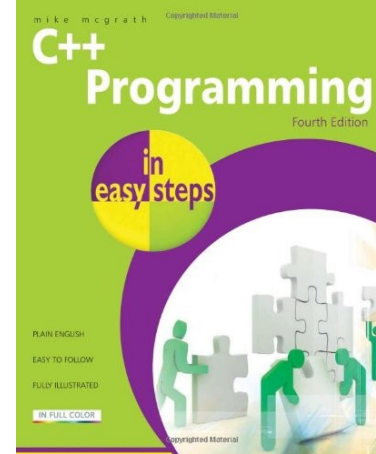
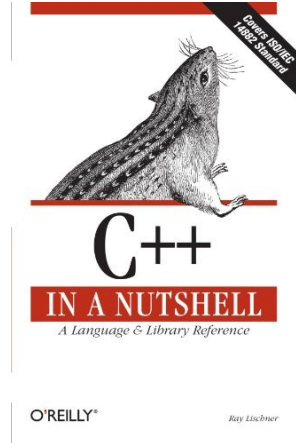
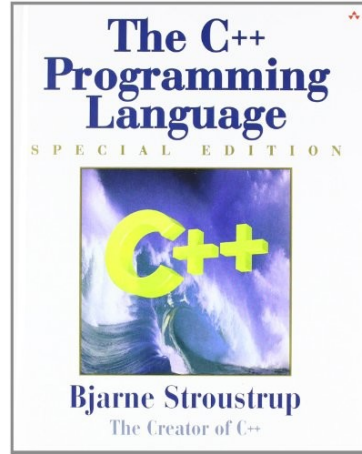
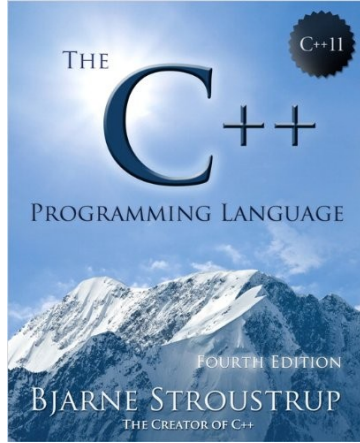
- **16 次课，共 32 学时，上机实验 16 学时；**

■ 成绩评定 (**课程组统一！**)

本门课程的评分分为 4 个部分，每个部分分数分配如下：课堂表现： 5 %；闭卷考试： 50%；平时作业 + 大程序： 45 %。

- **期末考试 50%；**
- **平时 50%**
 - **Pintia 作业 30%；**
 - **大作业 15%；**
 - **课堂表现 5%；（随堂点名 + pintia 平时作业综评）**

Useful Books on C++



参考教材

- Intruduction to Programming with C++ , Y.Daniel Liang , 机械工业出版社;
- Thinking in C++ , Bruce Eckel , 机械工业出版社;
- 面向对象程序设计高级教程 陈奇编著, 高等教育出版社;
- The C++ Programming Language , Bjarne Stroustrup , Addison-Wesley Publishing Company ;
- *C++ How to Program*- Fourth Edition, by H. M. Deitel, P. J. Deitel, Prentice Hall, New Jersey ;
- Programming: Principles and Practice Using C++, Ed.Bjarne Stroustrup , Addison-Wesley Publishing Company ;
- C++ primer ; Stanley B.Lippman, Josee Lajoie, Barbara E.Moo;
- 不限定……

C/C++ 编程技巧提升：那些值得收藏的资源网站

- <https://www.tutorialspoint.com/cplusplus/index.htm>
- <http://www.learncpp.com/>
- <http://www.cplusplus.com/>
- [https://www.cprogramming.com/tutorial/c++-tutorial.html/](https://www.cprogramming.com/tutorial/c++-tutorial.html)
- <http://www-h.eng.cam.ac.uk/help/tpl/languages/C++.html>
- <https://en.wikipedia.org/wiki/C%2B%2B>
- <https://isocpp.org>

Useful Links on C++

- C++ Programming Language Tutorials – C++ Programming Language Tutorials.

<http://www.cs.wustl.edu/~schmidt/C++/>

- C++ Programming – This book covers the C++ programming language, its interactions with software design and real life use of the language.

https://en.wikibooks.org/wiki/C++_Programming

- Free Country – The Free Country provides free C++ source code and C++ libraries for a number of C++ programming areas including compression, archiving, game programming, the Standard Template Library and GUI programming.

<https://www.thefreecountry.com/sourcecode/cpp.shtml>

- C and C++ Users Group – The C and C++ Users Group provides free source code from C++ projects in a variety of programming areas including AI, animation, compilers, databases, debugging, encryption, games, graphics, GUI, language tools, system programming and more.

<http://www.hal9k.com/cug/>

针对初学者

• 1. Learn C/C++

网址: [Learn C - Free Interactive C Tutorial \(learn-c.org\)](http://learn-c.org) 和 [Learn C++ - Free Interactive C++ Tutorial \(learn-cpp.org\)](http://learn-cpp.org)

简介: 这是一个提供免费在线交互式学习资源的网站, C 语言是一种广泛使用的编程语言, 特别适合系统编程和嵌入式系统。网站提供了一系列由浅入深的教学课程, 包括 C 语言的基础知识、数据类型、控制流、函数、指针、数组和结构体等主题。界面简洁, 便于用户自学, 还提供了在线的代码编辑器和执行环境, 让用户可以实时编写并测试代码。C++ 是一种面向对象的编程语言, 它基于 C 语言, 并增加了许多新的特性, 如类和对象、模板、异常处理、重载和继承等。和 learn-c.org 类似, learn-cpp.org 提供了不同难度的教程, 覆盖了 C++ 语言的核心概念, 并同样配有在线编写和运行代码的功能。用户可以通过逐步的教程, 加深对 C++ 编程语言的理解和应用能力。

• 2. C++ Tutorial

网址: [Learn C++ - Free Interactive C++ Tutorial \(cplusplus.com\)](http://cplusplus.com)

简介: 该站点为初学者和进阶学习者提供了 C++ 学习内容的逐步引导, 它涵盖了从基础语法到高级特性的各个方面。访问者可以找到有关 C++ 标准库的详细参考资料, 包括各种数据结构、算法、函数等。网站还设有关于 C++ 编程技巧和最佳实践的文章和讨论区。

3. TutorialsPoint-C/C++ Programming

网址: [C Tutorial \(tutorialspoint.com\)](http://tutorialspoint.com)

简介: 提供了完整的 C 语言教程和示例程序, 适用于理论学习和实践。

有一定基础的学习者

- 1. 菜鸟 C++ 教程

网址: C Tutorial ([runoob.com](https://www.runoob.com))

简介: 通俗易懂的语言来讲解 C++ 编程语言。

- **2. CPPReference**

网址: [cppreference.com](https://en.cppreference.com)

简介: 一个广泛使用的 C/C++ 参考手册, 包括标准库的各种功能和示例代码。

- **3. GeeksforGeeks**

网址: <https://www.geeksforgeeks.org/c-programming-language/>

简介: 包含了大量针对 C/C++ 的编程概念、数据结构、算法和面试题。

进阶或专业学习者

- **1 . Stackoverflow**

网址: <https://stackoverflow.com/questions/388242/the-definitive-c-book-guide-and-list>

简介: 这是 Stack Overflow 社区维护的 C++ 书籍推荐列表, 包括了从初学者到专家级的各种书目推荐。

- **2. Github**

这不是一个传统意义上的学习网站, 但通过阅读、参与开源项目和代码库, 可以学习到大量的高级 C++ 技术和实际的编码风格。

小结

除了以上网站，还有在线课程平台提供了由学术机构或行业专家创建的 C/C++ 相关课程。这些课程通常涵盖从基础知识到专业应用各个层面。

参与开源项目、阅读其他人的代码、学习 C/C++ 的最佳实践和设计模式，也是提高 C/C++ 水平的好方法。

C++ 面向对象程序设计 - 教学要求

• 课程基本要求

1. 掌握用 C++ 进行的面向对象编程的方法，能够运用 C++ 编写求解简单应用问题的面向对象程序；
2. 掌握 C++ 对 C 面向对象的语言扩展如常量、引用等。
3. 掌握类与封装、对象、继承、多态等面向对象的语言特征和概念，及其在 C++ 中的实现技术和方法；
4. 理解类与对象、构造函数、析构函数，类型转换、重载函数、虚函数、抽象类等面向对象程序设计中的重要概念及其实现方法；
5. 掌握面向对象的异常处理机制及 C++ 中的编程方法；
6. 理解函数模版、类模版，并用运用 C++ STL 编写程序；
7. 了解 MFC++ 和事件驱动编程机制（大作业！）。

目录 /

Contents



01

C++ 简介

02

C++ 面向对象程序设计

03

C++ 标准库

04

C++ 高效性

05

C++ 应用场景

06

C++ 常见编译器介绍

01



C++ 简介

- C++ 是由 C 语言扩展升级而来，最早于 1979 年由本贾尼·斯特劳斯特卢普在 AT&T 贝尔工作室研发。C++ 在继承了 C 语言过程化程序设计特性的基础上，进一步扩充和完善了 C 语言，引入了面向对象的程序设计概念，如抽象数据类型、继承和多态等。这使得 C++ 在面向对象程序设计方面具有强大的能力，同时它也可以进行基于过程的程序设计。



C++ 语言之父：本贾尼·斯特劳斯特卢普 **Bjarne**



- 起源（1979 年）：C++ 的历史可以追溯到 1979 年，当时 **Bjarne Stroustrup** 在准备他的博士毕业论文时，使用了 Simula 语言，并受到其面向对象特性的启发。
- "C with Classes"（1980 年代初）：Stroustrup 开始着手将面向对象的概念引入 C 语言，最初称之为 "C with Classes"。
- C++ 命名（1983 年）：该语言在 1983 年被正式命名为 C++，名称来源于 C 语言的 "++" 运算符，象征语言的递增和改进。
- 《The C++ Programming Language》出版（1985 年）：Stroustrup 的 C++ 参考手册《C++ Programming Language》出版，同年，C++ 的商业版本问世。



- 标准化过程（1989 年）：C++ 的标准化工作开始，成立了 ANSI 和 ISO 的联合标准化委员会。
- C++98（1998 年）：发布了 C++ 语言的第一个国际标准—ISO/IEC 14882:1998，即 C++98。
- C++03（2003 年）：针对 C++98 存在的问题进行了修订，发布了 C++03 标准。
- C++11（2011 年）：引入了大量现代编程特性，如自动类型推断、基于 lambda 的表达式、标准线程库等。



- C++14（2014 年）：作为 C++11 的增量更新，支持普通函数的返回类型推演，泛型 lambda 等。
- C++17（2017 年）：新增了 UTF-8 字符文字、折叠表达式、结构化绑定等特性。
- C++20（2020 年）：引入了模块、协程、范围、概念与约束等新特性，是自 C++11 以来最大的发行版。

传统 C++ 与标准 C++

- 三次主要订：1985 年，1990 年，1998 年，2011 年（ C++ 11 ）
 - 1998 年确定的版本为标准 C++，之前的称传统 C++。
 - 标准 C++ 包括传统 C++ 的全部功能，且更庞大、全面
 - .h 和无扩展名的头文件：
 - 传统 C++ 为 .h 头文件，如 iostream.h、fstream.h、string.h；
 - 标准 C++ 对应的头文件为 iostream、fstream、string。

标准 C++ 程序和传统 C++ 程序对比

//Eg1-2.cpp :传统C++

#include <iostream.h>

#include <stdio.h>

#include <math.h>

void main() {

int x;

cout<<"输入数字: ";

scanf("%d", &x);

bool prime=true;

for(int i=2;(i<=x-1)′i++)

if(x%i==0) prime=false;

if(prime)

cout<< x<<"是素数!"<<endl;

else

cout<<x<<"不是素数!"<<endl;

}

库函数
头文件
差异

//Eg1-2.cpp :标准C++

#include <iostream>

#include <cstdio>

#include <cmath>

void main() {

int x;

std::cout<<"输入数字: ";

scanf("%d", &x);

bool prime=true;

for(int i=2;(i<=x-1)′i++)

if(x%i==0) prime=false;

if(prime)

std::cout<< x<<"是素数!"<<std::endl;

else

std::cout<<x<<"不是素数!"<<std::endl;

}

库函数引
用差异

- C++ 的发展受到了多种编程语言的影响，包括 Simula、Algol68、BCPL、Ada 等，它不仅继承了 C 语言的高效性和灵活性，还加入了面向对象、泛型编程和异常处理等特性。C++ 的设计哲学是提供给程序员强大的功能，同时保持与 C 语言的兼容性。
- C++ 语言的发展历程显示了它如何逐渐演化成为一个功能强大、应用广泛的编程语言，它在系统软件、游戏开发、嵌入式系统等领域都有广泛的应用。随着新标准的发布，C++ 不断引入创新特性，以适应现代软件开发的需求。

02



C++ 面向对象程序设计

面向过程是一种计算机编程思想，它主要关注程序的执行流程和数据处理，强调程序由一系列相互依赖的函数或子程序组成，并且这些函数按照特定的顺序执行以完成特定的任务。在面向过程编程中，将问题分解为多个小部分，并对每个小部分进行具体实现和优化，最终整合起来形成一个完整的程序。

面向过程编程通常包括以下步骤：

- **定义问题**：确定要解决的问题或任务；
- **分析问题**：将问题分解为多个小部分并确定其相互依赖关系；
- **设计方案**：根据需求选择适当的数据结构和算法，并定义各个子程序或函数；
- **编写代码**：实现各个子程序或函数，并按照设计方案组合起来形成完整的程序；
- **调试测试**：对程序进行测试、调试和优化，确保其能够正确地运行。

面向过程编程思想的核心：**功能分解、自顶向下、逐层细化**（**程序=数据结构+算法**）。

面向过程的主要缺点是不符合人的思维习惯、重用性低、维护困难等。

面向对象编程（ Object Oriented Programming ，简称 OOP ）是一种编程思想，它将程序看作一个由各种独立的对象组成的集合。每个对象都有自己的属性和方法，并且可以与其他对象进行交互，共同完成任务。在面向对象编程中，通过类和实例来描述具体事物，类定义了一类事物的通用属性和方法，而实例则是具体某个事物的一个具体表现。

面向过程与面向对象程序设计

面向对象程序设计基本概念

— 对象

- 客观存在的实体称为对象

— 属性

- 描述对象的特征的数据

— 行为

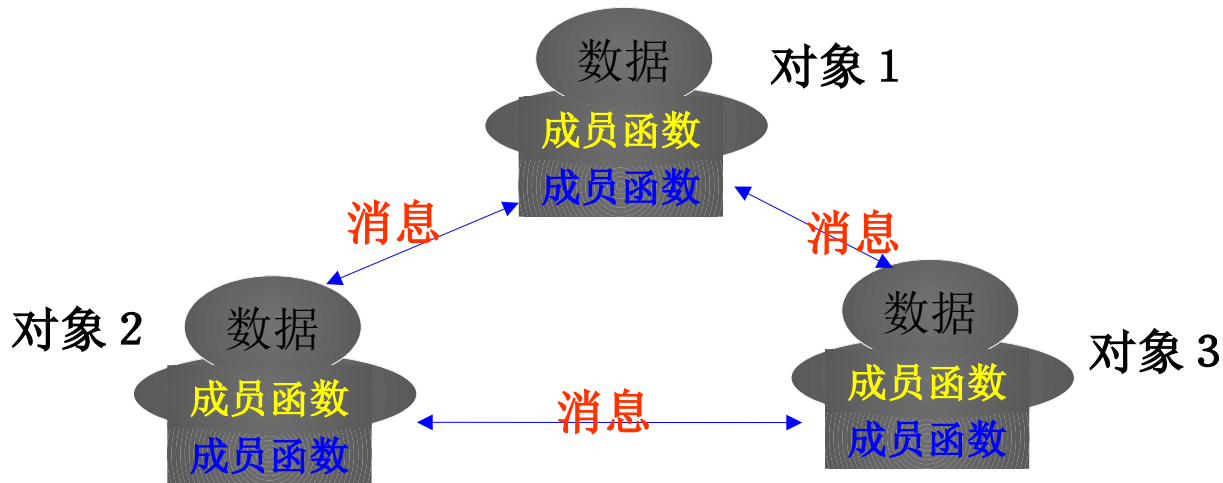
- 对象自身的行为，对现实世界某些信息的应。

— 消息

- 对象之间通过传递消息相互影响，消息可简单理解为传递给被调用函数的**实参**。

面向对象程序设计

面向对象的程序范型



- 将客观事物的属性和行为抽象成数据和操作数据的函数，并把它们组合成一个不可分割的整体（即对象）的方法能够实现对客观世界的真实模拟，反映出世界的本来面目。从客观世界中抽象出一个个对象，对象之间能够传递消息。

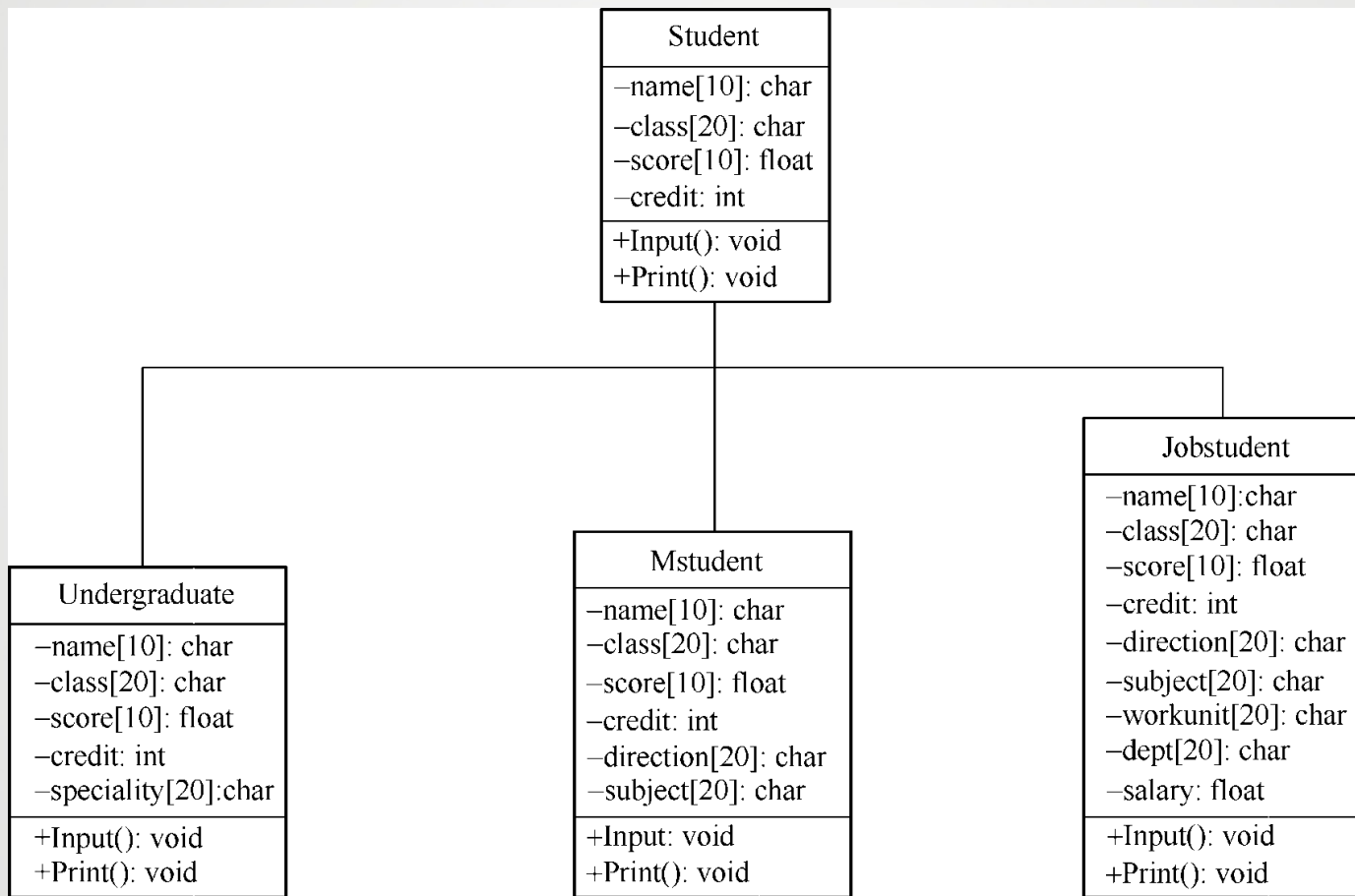
➤➤ C++ 面向对象程序设计的特性主要包括以下四个方面 :26

- 抽象 (**Abstraction**) : 抽象是面向对象方法的核心特性, 它涉及到对具体问题或对象进行概括, 抽出一类对象的公共性质并加以描述的过程。在C++中, 抽象可以通过定义类来实现, 类中包含**数据成员 (属性)**和**成员函数 (行为)**。数据成员描述了对对象的静态特征, 而成员函数则定义了对对象可以执行的操作。
- 封装 (**Encapsulation**) : 封装是将抽象得到的数据和行为相结合, 形成一个有机整体的过程。在C++中, 这通常通过类来实现, 类将数据和操作数据的函数代码进行有机的整合, 形成一个独立的实体。封装有助于隐藏对象的内部细节, 只通过明确的接口与外部交互, 从而提高了代码的安全性和可维护性。

➤➤➤ C++ 面向对象程序设计的特性主要包括以下四个方面 :27

- 继承 (**Inheritance**) : 继承是面向对象编程中的一个重要概念, 它允许创建一个新类 (派生类) 继承自一个现有类 (基类)。派生类继承了基类的所有属性和方法, 并可以添加新的属性和方法或重写现有方法。通过继承, 可以实现代码的重用和扩展, 提高了软件开发的效率。
- 多态 (**Polymorphism**) : 多态是对人类思维方式的一种直接模拟, 它允许同一个操作作用于不同的对象时, 产生不同的结果。在 C++ 中, 多态可以通过虚函数和动态绑定来实现。当派生类重写了基类的虚函数时, 根据对象的实际类型来调用相应的函数实现, 这就是多态的体现。多态提高了代码的灵活性和可扩展性。

>>> C++ 面向对象程序设计的特性主要包括以下四个方面 :28



03



C++ 标准库

- C++ 标准库 (C++ Standard Library) 是 C++ 编程语言的一个组成部分，它包含了一组预定义的函数、对象和类，这些都可以被 C++ 程序直接使用。标准库提供了广泛的功能，从基本的数据结构（如数组、向量和列表）到复杂的算法（如排序和搜索），再到输入 / 输出操作、文件操作、数学函数、时间日期处理等等。

»» C++ 标准库可以分为以下几类：

31

- **C 库**：这部分由 C 标准库扩展而来，它强调结构、函数和过程，并不支持面向对象技术。
- **标准的 C++ 库**：这包括 C++ I/O 库、String 库、时间库（ chrono ）、异常库（ exception ）、数值库等，为 C++ 提供基本的输入 / 输出、字符串处理和数值计算功能。
- **标准模板库（ STL ）**：STL 是 C++ 标准库中最重要的一部分，它包含了诸多在计算机科学领域里所常用的基本数据结构和基本算法。STL 库主要包括六大组件，分别是容器（ Containers ）、算法（ Algorithms ）、迭代器（ Iterators ）、配接器（ Adapters ）、函数对象（ Function Objects ）和分配器（ Allocators ）。其中，容器用于管理某一类对象的集合，如向量（ vector ）、链表（ list ）、队列（ queue ）、栈（ stack ）、集合（ set ）和映射（ map ）等；算法提供了各种常用算法的实现，如排序、查找、合并、计数等；迭代器定义了一种通用的遍历容器元素的接口，使得可以使用相同的代码处理不同类型的容器。

- 容器 (Containers) : 如向量 (vector)、列表 (list)、集合 (set)、映射 (map) 等，这些数据结构用于存储和管理数据。
- 算法 (Algorithms) : 如排序 (sort)、搜索 (search)、查找 (find) 等，这些算法可以在容器上操作数据。
- 迭代器 (Iterators) : 用于遍历容器中的元素。
- 输入 / 输出流 (I/O Streams) : 如 cin 和 cout，用于从控制台读取数据和向控制台写入数据。
- 字符串处理 (String Processing) : 如 std::string 类，用于处理字符串。
- 数学函数 (Mathematical Functions) : 如平方根 (std::sqrt)、正弦 (std::sin) 等。
- 时间和日期 (Time and Date) : 如 std::chrono 库，用于处理时间和日期。

04



C++ 高效性

- **贴近硬件**：C++ 提供了直接访问内存和位操作的能力，允许程序员进行底层编程，从而实现对硬件资源的精细控制。
- **性能优化**：C++ 编译器采用了多种优化技术，如内联函数、循环展开、常量传播等，以提高程序的执行效率。
- **无垃圾收集**：C++ 不包含自动垃圾收集机制，而是依赖于程序员或智能指针等自动存储管理技术来管理内存，这有助于避免垃圾收集可能带来的性能开销。
- **丰富的标准库**：C++ 标准库提供了高效的数据结构和算法实现，如排序、搜索等，这些实现已经过优化，可以直接用于提高程序效率。

- **模板编程**：C++ 的模板允许在编译时进行多态，这意味着模板代码可以根据不同的数据类型进行优化，从而提高运行时效率。
- **零成本抽象**：C++ 的设计哲学之一是使得抽象化对性能的影响尽量减小，即所谓的“零成本抽象”。这使得开发者可以使用面向对象等高级特性而不会显著牺牲性能。
- **多线程支持**：C++11 及后续标准提供了多线程库，允许开发者利用多核处理器的计算能力，进一步提高程序的执行效率。
- **现代 C++ 特性**：C++11 及以后的标准引入了许多现代编程语言特性，如基于自动类型推断的 auto 关键字、基于范围的 for 循环、右值引用等，这些特性在保持代码清晰和简洁的同时，也有助于提高程序的运行效率。

- **内存管理**：C++ 提供了多种工具和技术用于高效的内存管理，包括构造函数、析构函数、智能指针等，这些机制帮助避免内存泄漏和野指针，同时减少不必要的内存分配和释放操作。
- **避免不必要的复制**：C++ 提供了对象的移动语义，允许在不牺牲性能的情况下，通过移动而不是复制来传递大型对象或资源。
- **编译时多态**：通过模板和内联函数，C++ 能够实现编译时多态，这通常比运行时多态（如虚函数）更高效。
- **低级别系统编程**：C++ 常用于编写操作系统、驱动程序、嵌入式系统等对性能要求极高的应用。

- **跨平台特性**：C++ 程序可以在多种硬件和操作系统平台上编译运行，而不需要或仅需很少的修改，这使得 C++ 程序能够充分利用不同平台的硬件特性。
- **工具链和社区**：C++ 拥有一个成熟的工具链，包括调试器、性能分析工具等，以及一个活跃的社区，这些都有助于开发者编写和优化高效的 C++ 程序。

05



C++ 应用场景

主要的 C++ 应用场景：

1/3

39

- **系统软件开发**：C++ 常用于开发操作系统、文件系统、驱动程序等系统软件，因为这些软件要求与硬件接近，同时对性能有较高要求。
- **游戏开发**：游戏引擎和游戏客户端通常使用 C++ 进行开发，以利用其高性能特性，尤其是在图形渲染、物理模拟和人工智能等方面。
- **嵌入式系统开发**：C++ 因其性能优势和对资源的精细控制，被广泛应用于嵌入式系统开发，包括物联网 (IoT) 设备、汽车电子、工业控制系统等。
- **服务器端开发**：对于需要处理大量并发请求和数据的后端服务，C++ 提供了高效的网络通信库和多线程支持，使其成为开发高性能服务器程序的选择。

主要的 C++ 应用场景：

2/3

40

- **高性能计算**：在科学计算和大规模数值分析领域，C++ 的高性能特性使其成为实现复杂算法和数据处理任务的理想选择。
- **图形和视觉处理**：C++ 在图形渲染、图像处理、计算机视觉和虚拟现实等领域有广泛应用，尤其是在需要实时处理大量数据的场合。
- **客户端开发**：桌面应用程序，如办公软件、媒体播放器和专业软件，常使用 C++ 进行开发，以提供快速响应和良好的用户体验。
- **游戏引擎开发**：许多流行的游戏引擎，如 Unreal Engine，使用 C++ 构建，以确保性能和对底层系统的控制。

主要的 C++ 应用场景：

3/3

41

- **人工智能和机器学习**：尽管 Python 在 AI 领域更为流行，但 C++ 在实现高性能的数学库、深度学习框架的底层以及实时系统方面仍然发挥着重要作用。
- **音视频处理**：C++ 在音视频编解码、流媒体处理和实时通信协议开发中扮演着关键角色，如 FFmpeg 和 WebRTC 等项目就是用 C++ 编写的。
- **金融和高频交易**：在对性能和实时性要求极高的金融领域，C++ 被用于开发高频交易系统和金融分析工具。
- **网络编程**：C++ 适用于服务器端应用程序、网络管理和网络通信等领域，可提供更高的运行速度和性能。
- **数据库引擎**：许多数据库引擎，如 MySQL 和 MongoDB，都是用 C++ 编写的。这得益于 C++ 对低级系统的访问能力和性能优化。

06



C++ 常见编译器介绍

一些广泛使用的 C++ 编译器：

43

- GNU Compiler Collection (GCC)：GCC 是一个自由软件编译器集合，支持 C、C++、Objective-C、Fortran、Ada 和 Go 等多种编程语言。它是最流行的开源 C++ 编译器之一，广泛用于各种操作系统和平台上。
- Clang：Clang 是基于 LLVM 的 C、C++、Objective-C 和 Objective-C++ 编译器前端。它以其出色的错误消息、性能以及丰富的源代码静态分析工具而闻名。
- Microsoft Visual C++ (MSVC)：MSVC 是微软开发的 C++ 编译器，与 Visual Studio 集成。它为 Windows 平台提供了强大的开发工具，包括集成开发环境、调试器和性能分析工具。
- Intel C++ Compiler (ICC)：由 Intel 开发的 C++ 编译器，提供了自动向量化和并行化等高级优化特性，特别适用于需要高性能计算的应用。

一些广泛使用的 C++ 编译器：

44

- Borland C++ ： Borland 系列编译器（包括早期的 Turbo C++ ）曾经非常流行，尤其是在 DOS 和 Windows 3.1 时代。 Borland C++ Builder Compiler 是其中的一个产品，它包括了对 ANSI/ISO C++ 语言的支持。
- Watcom C++ ： Watcom C++ 是一个历史悠久的编译器，以其在 DOS 时代的高效编译而著称。现在， Watcom C++ 作为 Open Watcom 开源包发行，支持多种操作系统。
- Digital Mars C++ Compiler ： Digital Mars 是一款高性能的 C 和 C++ 编译器，它包括了一个集成开发环境和多种编程工具，如反汇编器和资源编译器。
- Tiny C Compiler (TCC) ： TCC 是一个小型的 C 语言编译器，虽然它主要用于 C 语言，但也支持一些 C++ 特性。
- Code::Blocks ： Code::Blocks 是一个轻量级的 C++ 集成开发环境，它支持多种编译器，包括 GCC 和 Clang ，并且提供了代码自动补全、调试和版本控制等功能。
- Qt Creator ：基于 Qt 框架的 C++ 集成开发环境，提供了一套完整的工具集，用于开发跨平台的应

一些广泛使用的 C++ 编译器：

45

- 这些编译器各有特点，其中，GCC 和 Clang 因其开源和跨平台特性在许多 Linux 发行版中被广泛采用；而 MSVC 则在 Windows 平台上提供了优秀的集成开发体验。

- g++ 和 gcc 是 GNU 编译器集合（ GNU Compiler Collection ，简称 GCC ）中的两个相关但独立的编译器，它们都用于从源代码生成可执行程序，不过它们分别针对不同的编程语言。
- gcc：
 - gcc 是 GNU C Compiler 的缩写，它是 GCC 中用于编译 C 语言程序的编译器。
 - 它支持多种与 C 语言相关的语言标准，如 ANSI C（ ISO C89 ）、 ISO C99、 ISO C11 等。
 - gcc 也可以用于编译 C++ 程序，但需要链接 C++ 标准库。在某些系统中，可能存在一个名为 c++ 的符号链接到 g++，这样你就可以使用 c++ 命令来编译 C++ 程序。
- g++：
 - g++ 是 GNU C++ Compiler 的缩写，它是 GCC 中用于编译 C++ 程序的编译器。
 - 它不仅包含了 gcc 的功能，还增加了对 C++ 特定特性的支持，如类、模板、异常处理等。
 - g++ 默认链接 C++ 标准库（如 libstdc++），而 gcc 默认链接 C 标准库。

➤ 它们之间的具有以下关系：

相同之处：

- 它们都是 GCC 编译器集合的一部分。
- 它们共享底层的优化器和代码生成器，因此对于底层的 C 代码，两者在优化和生成的机器代码方面可能是相同的。

不同之处：

- gcc 主要用于编译 C 语言程序，而 g++ 用于编译 C++ 程序。
- g++ 在编译 C++ 程序时会启用 C++ 语言特性，并链接 C++ 标准库，而 gcc 需要额外指定链接 C++ 标准库的选项（如 `-lstdc++`）。
- g++ 支持 C++ 特有的编译选项和编译阶段，如模板元编程、名称修饰（`name mangling`）和 RTTI（运行时类型识别）。

C++ 程序的结构

1、C++ 程序的构成

- 声明部分
- 主函数部分
- 函数定义

2、C++ 程序文件

- 头文件：.h .hpp
- 源文件：.cpp, .cc

»» C++ 第一份代码 "hello,world" :

49

- VS Code 编写 "hello, world" , 源代码文件名 helloworld.cpp

```
#include <iostream>
```

```
int main()
```

```
{ // 输出 "Hello, World!"
```

```
    std::cout << "Hello, World!" << std::endl;
```

```
    return 0;
```

```
}
```

»» C++ 第一份代码 "hello,world" :

50

Linux 环境下，使用 g++ 编译并生成 helloworld 的可执行程序：

```
g++ -o helloworld helloworld.cpp
```

Linux 环境下，执行 helloworld 可执行程序

```
helloworld
```

- 预处理指令 `#include <iostream>`
 - `#include` 是一个预处理指令，它告诉编译器在编译之前要包含指定的头文件。
 - `<iostream>` 是一个标准库头文件，它包含了 C++ 标准库中关于输入 / 输出流的对象和函数的声明。`<iostream>` 提供的 `std::cout` 对象，它代表标准输出流，用于向控制台输出数据。
- 主函数 `int main()`
 - `main` 函数是 C++ 程序的入口点，即程序从这里开始执行。
 - `int` 是 `main` 函数的返回类型，表示 `main` 函数将返回一个整数。
 - 在操作系统中，这个返回值通常用于指示程序是否成功执行。通常，返回 0 表示程序成功执行，而非零值表示出现错误。

- 输出语句 `std::cout << "Hello, World!" << std::endl;`
 - `std::cout` 是一个输出流对象，用于向标准输出（通常是控制台或终端）发送数据。
 - `<<` 是一个插入运算符，用于将数据发送到输出流。
 - `"Hello, World!"` 是一个字符串字面量，表示要输出的文本内容。
 - `std::endl` 是一个流操纵符，它向输出流插入一个换行符，并刷新输出缓冲区，确保数据立即被显示在控制台上。
- 返回值 `return 0;`
 - `return` 语句用于从函数中返回一个值。
 - 在 `main` 函数中，`return 0;` 表示程序成功执行，并返回状态码 0。

【例 1-1】创建一个控制平台应用程序，当其运行时在屏幕上显示“我们欢迎你”。

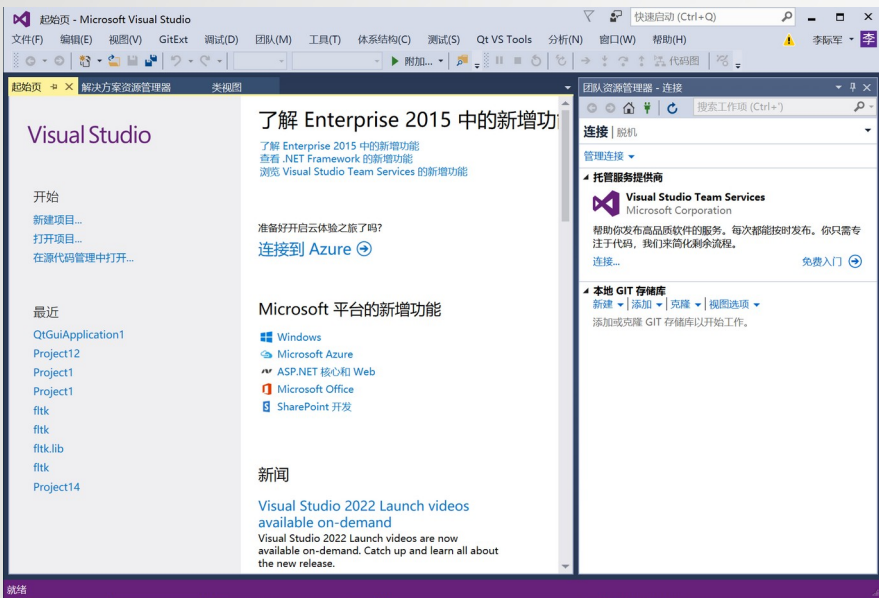
01

OPTION

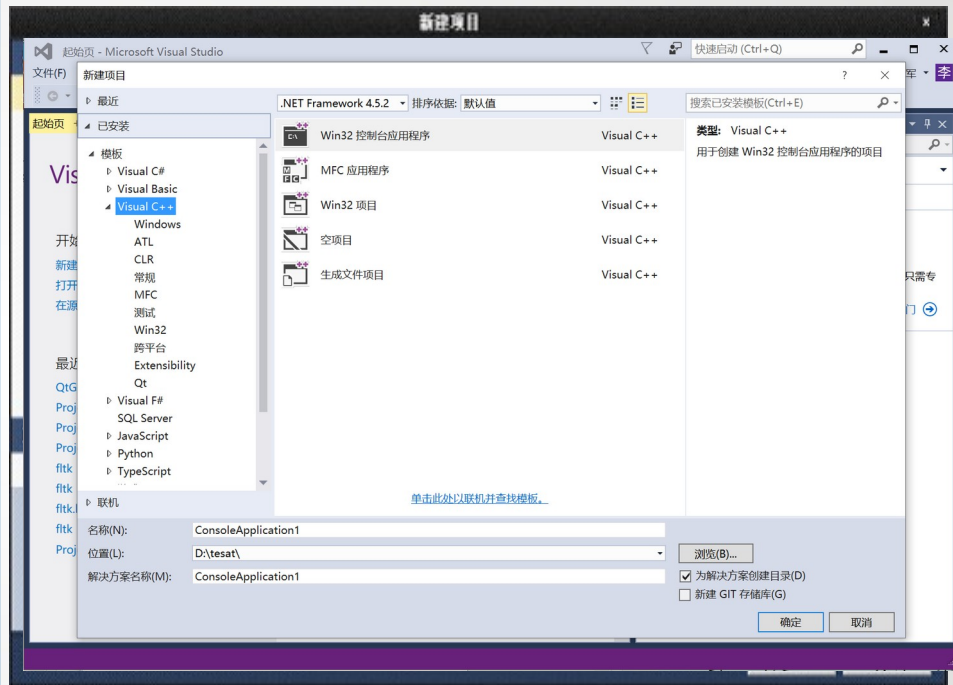
建立一个项目

(Project)

在 Microsoft Visual Studio 201X 下开发程序时，首先要创建一个项目。项目中存放了建立程序所需要的全部信息。启动 Microsoft Visual Studio 201X；“文件”菜单上一次单击“新建”→“项目”菜单项。

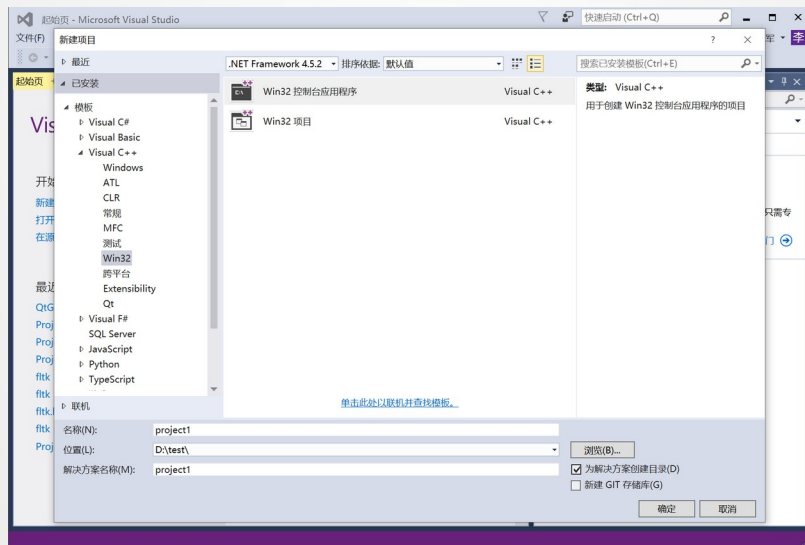


启动 Microsoft Visual Studio 2015



新建项目

选择“已安装模版”下的“Visual C++”的“Win32”，再选择对话框中间的“Win32 控制台应用程序”，然后在项目“名称”字段中输入“Project1”，在项目“位置”字段中输入要保存项目的位置。



单击“确定”按钮出现“Win32 应用程序向导”窗口，在“Win32 应用程序向导”窗口中单击“完成”按钮。

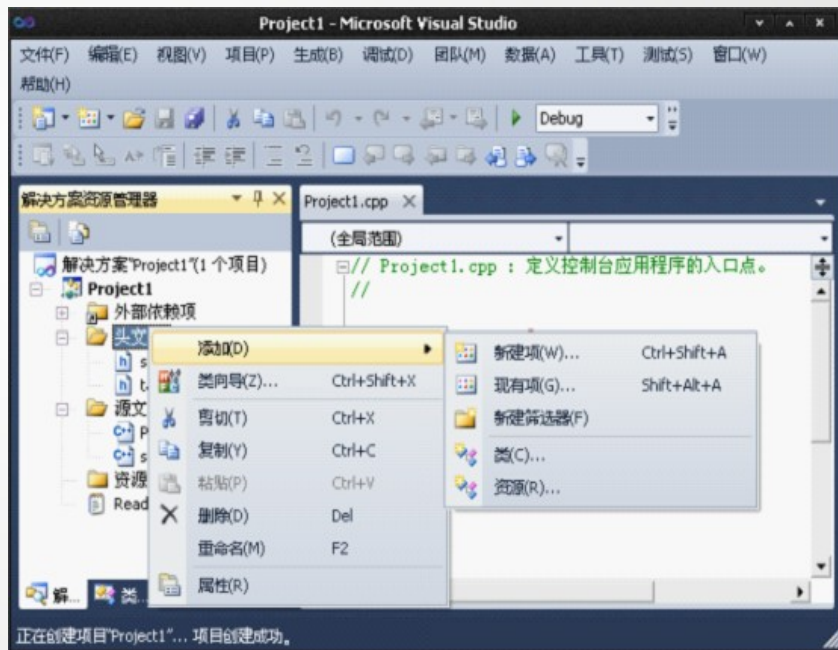


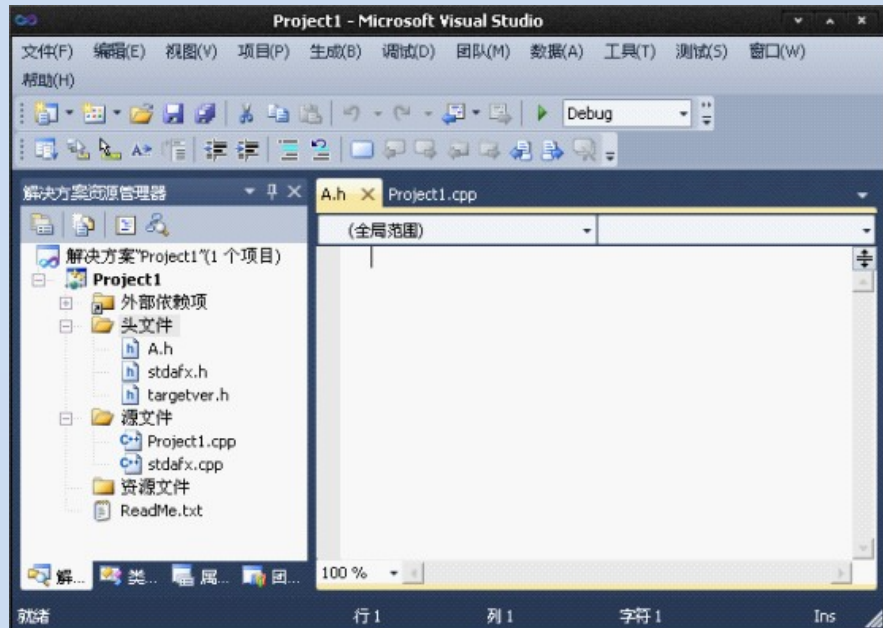
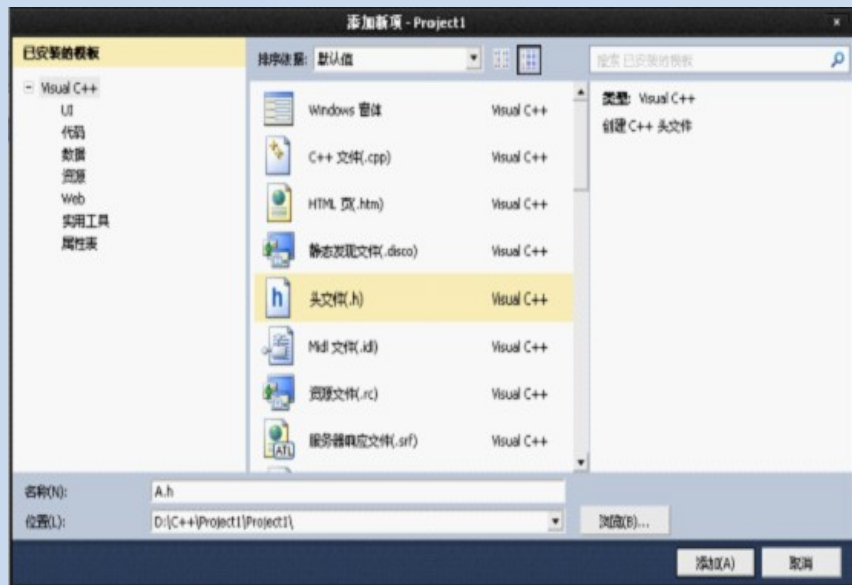
02

OPTION

创建类

创建类 A，在图中选择“头文件”，在弹出的快捷菜单中选择“添加”→“新建项”，弹出“添加新项”对话框，在对话框中选择“头文件（.h）”，在名称中输入“A”。单击“添加”按钮，建立 A.h 头文件。





03

OPTION

编辑 A.h 文件

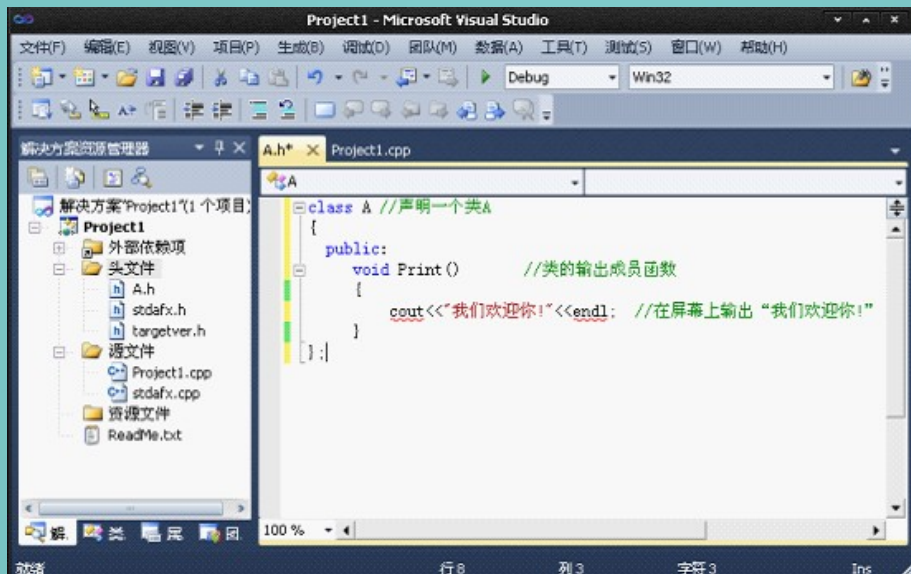
在类 A.h 的编辑区的空白区域中输入如下代码。

```
class A    // 声明一个类 A
{
    public:
    void Print()    // 类的输出成员函数
    {
        cout<<" 我们欢迎你 !" <<endl;// 在屏幕上输出 "我们欢迎你 !"
    }
};
```

03

OPTION

编辑 A.h 文件



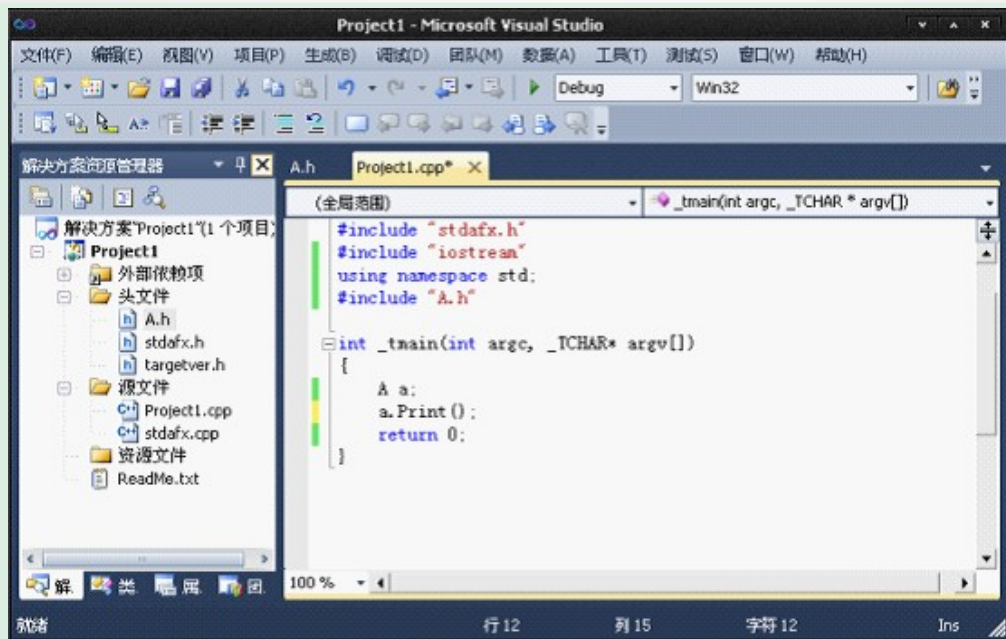
在 “Project1.cpp” 文件中输入如下代码。

```
#include <iostream>
using namespace std;
#include "A.h"
int main()
{
    A a;
    a.Print();
    return 0;
}
```

04

OPTION

编辑 Project1.cpp 文件

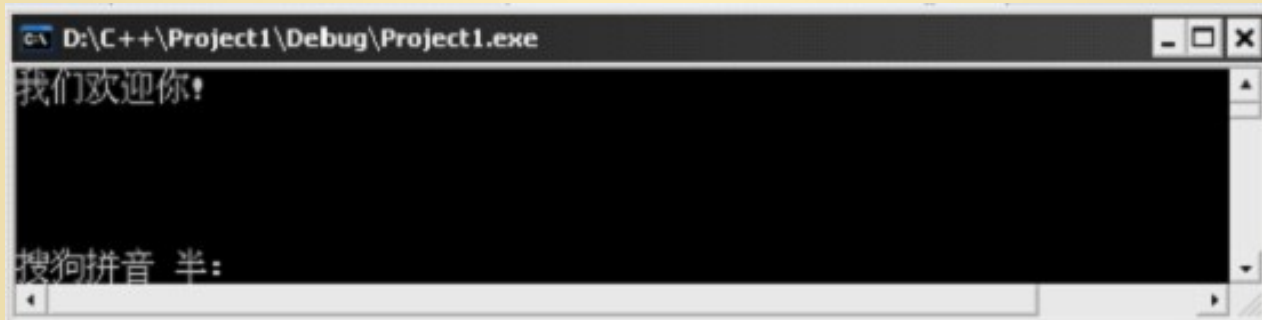


05

OPTION

运行

执行“调试”→“开始执行（不调试）”命令或 Ctrl+F5 组合键，进行程序的编译、链接和运行，运行结果如图所示。



07



数据输入与输出

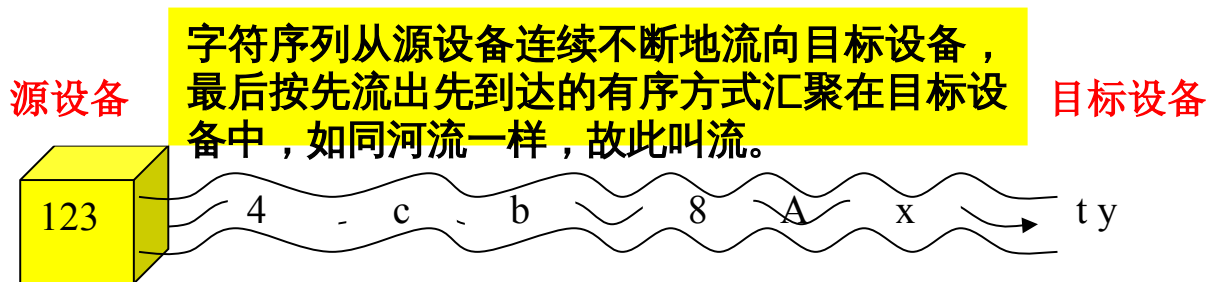
C++ 的数据类型

- 基本类型
 - int, char, long, wchar_t, char16_t, char32_t
- 实型
 - float, double, long double
- 逻辑类型和空类型
 - bool, void
- 自定义类型
 - struct, class, union, enum, 指针, 数组
- STL 中的类型
 - vector, string, list, stack, map, set.....
- C++11 : long long

流的概念

C 及 C++ 中的流概念

- I/O (input/output , 输入 / 输出) 数据是一些从源设备到目标设备的字节序列，称为字节流。除了图像、声音数据外，字节流通常代表的都是字符，因此在多数情况下的流 (stream) 是从源设备到目标设备的字符序列，



流的概念

— 输入流 :istream

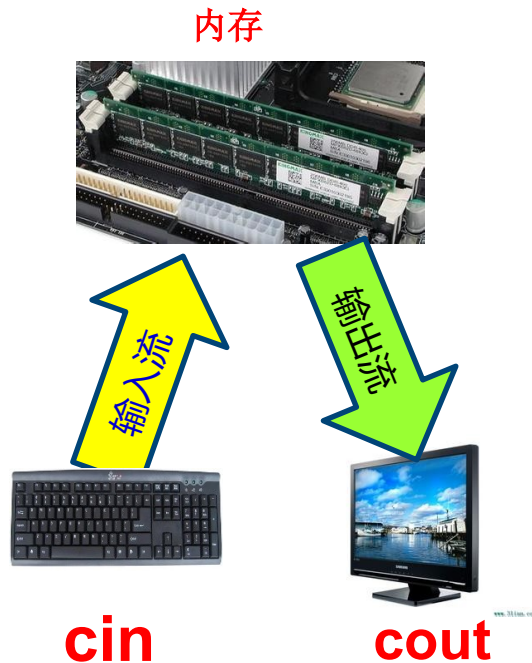
- 输入流 (input stream) 是指从输入设备流向内存的字节序列。

— 输出流 :ostream

- 输出流 (output stream) 是指从内存流向输出设备的字节序列。

— C++ 中的输入输出流

- `iostream` : `istream`, `ostream`
- `cin` 输入流对象, C++ 已将其与键盘关联
- `cout` 输出流对象, C++ 已将其与显示器关联



cin 和提取运算符 >>

1、cin 的用途

- cin 是 iostream 中用 istream 定义的一个输入流对象对象，定义类似于：

istream cin;

- 在 C++ 程序，cin 主要用于从键盘输入数据，当然也可以使用 scanf 函数，但 cin 更简单。

2、cin 的用法

- 输入单个变量的值

cin>>x ;

- 输入多个变量的值

cin>>x1>>x2>>x3>>x4.....>>xn ;

其中 x,x1.....x2 可是以内置数据类型如 int , char , float , double 等。

cin 和提取运算符 >>

用 cin 时的注意事项

- cin 具有自动识别数据类型的能力
- 在一条 cin 语句中同时为多个变量输入数据。输入数据的个数应当与 cin 语句中变量个数相同，各输入数据之间用一个或多个空白（包括空格、回车、Tab）作为间隔符，全部数据输入完成后，按 Enter 键结束。
- 在 >> 后面只能出现变量名，下面的语句是错误的。

```
cin>>"x">>x;    // 错误，>> 后面含有字符串 "x="
```

```
cin>>12>>x; // 错误，>> 后面含有常数 12
```

```
cin>>'x'>>x;
```

cout 和插入运算符 <<

1、cout 的用途

- cout (读作 see-out) 是 iostream 库中用 ostream 定义的一个输出流对象，类似于：

`ostream cout;`

- cout 已被 C++ 默认关联到显示器，用于在屏幕上输入数据。在 C++ 程序中，也可使用 C 语言的 sprintf 输出数据，但 cout 更简单。

2、cout 的用法

`cout<<x;`

- 其中 x 可是以内置数据类型如 int , char , float 等。

cout 和插入运算符 <<

3. 连续输出

- cout 能够同时输出多个数据，用法如下：

```
cout<<x1<<x2<<x3<<...;
```

例：

```
cout<<ch1<<ch2<<"C"<<"Hellow everyone!";
```

- 与 C 一样，在 C++ 中也可以将一条命令写在多行上。比如，上面的语句也可写成下面的形式：

```
cout<<ch1
```

```
<<ch2
```

```
<<"C"
```

```
<<"Hellow everyone!";
```

cout 和插入运算符 <<

4、输出换行

在 cout 语句中换行可用： “ \n” 或 endl

```
#include<iostream>
void main(){
    char ch1='c';
    char ch2[]="Hellow C++!";
    std::cout<<ch1<<std::endl;
    std::cout<<ch2<<"\n";
    std::cout<<"C"<<endl;
    std::cout<<"Hellow everyone!\n";
}
```


cout 和插入运算符 <<

4、输出数据间隔符

在连续输入多个数据时，应注意在数据之间插入间隔符。如

```
int x1=23;
```

```
float x2=34.1;
```

```
double x3=67.12;
```

```
cout<<x1<<x2<<x3<<900;
```

- 其中的 cout 语句将在屏幕上输出，
2334.167.12900

谁知道这是个什么数据呢？



名称	字符形式	值
空字符 (Null)	\0	0X00
换行 (NewLine)	\n	0X0A
换页 (FormFeed)	\f	0X0C
回车 (Carriage Return)	\r	0X0D
退格 (BackSpasc)	\b	0X08
响铃 (Bell)	\a	0X07
水平制表 (Horizontal Tab)	\t	0X09
垂直制表 (Vertical Tab)	\v	0X0B
反斜杠 (backslash)	\\	0X5C
问号 (question mark)	\?	0X3F
单引号 (single quote)	\'	0X27
双引号 (double quote)	\"	0X22



控制符	功能
endl	输出一个换行符，并清空流
ends	输出一个空字符，并清空流
dec	用十进制表示法输入或输出数值
hex	用十六进制表示法输入或输出数值
oct	用八进制表示法输入或输出数值
setfill (char <i>c</i>)	设置填充符 <i>c</i>
setprecision (int <i>n</i>)	设置浮点数输出精度 (包括小数点)
setw (int <i>n</i>)	设置输出宽度

输出格式控制符

- **1 . 设置浮点数的精度**

`setprecision(n)` // 最后一有效位将对其右边数字 4 舍 5 入
`cout<<setprecision(3)<<3.14126<<" "<<2.4576<<endl;`

输出：3.14 2.46

- **2 . 设置输出域宽和对齐方式**

`setw(n)`

- **3 . 设置对齐方式**

- `Setiosflags(long f);`
- `resetiosflags(long f);`

输出格式控制符

【例】用 `setiosflags` 和 `resetiosflags` 设置和取消输出数据的对齐方式。

```
#include<iostream>
#include<iomanip>
void main(){
    std::cout<<"123456781234567812345678"<< std:: endl;
    std:: cout<<setiosflags(ios::left)<<setw(8)
                <<456<<setw(8)<<123<< std::endl;
    std:: cout<<resetiosflags(ios::left)<<setw(8)
                <<123<< std:: endl;
}
```

输出格式控制符

4. 输出填充字符 (用指定字符填充空白)—— 补充内容

```
std::cout.fill(ch);  
std::cout<< std::setfill(ch);
```

【例】用 fill 和 setfill 设置输出填充字符。

```
//ch1-fill.cpp  
#include<iostream>  
#include<iomanip>  
void main()  
{  
    std::cout<<"123456781234567812345678"<<std::endl;  
    std::cout<<std::setw(8)<<123<<std::setw(8)<<456<<std::setw(8)<<789<<std::endl;  
    std::cout.fill('@');  
    std::cout<<std::setw(8)<<123<<std::setw(8)<<456<<std::setw(8)<<789<<std::endl;  
    std::cout<<std::setfill('^');  
    std::cout<<std::setw(8)<<123<<std::setw(8)<<456<<std::setw(8)<<789<<std::endl;  
}
```

数制基数

- 数制基数操纵符（在 `iostream` 头文件中定义）
 - `hex` : 16 进制 , `oct` : 8 进制 , `dec` : 10 进制
- 输入不同进制的数据
 - 在 `cin` 输入流中先插入数制操纵符 , 再输入数据
- 输出不同进制的数据
 - 在 `cout` 输出流中先插入数制操纵符 , 再输出数据
- 注意
 - 数制基数设置后将一致有效 , 直到下一次设置新数制基数才取消。

数制基数

```
#include<iostream>
using namespace std;
void main(){
    int x=34;
    cout<<hex<<17 <<" "<<x<<" "<<18<<endl;
    cout<<17 <<" "<<oct <<x<<" "<<18<<endl;
    cout<<dec<<17 <<" "<<x<<" "<<18<<endl;
    int x1, x2, x3, x4;
    cout<<" 输入  x1(oct), x2(oct), x3(hex), x4(dec):"<<endl;
    cin>>oct>>x1; // 八进制数
    cin>>x2;      // 八进制数
    cin>>hex>>x3; // 输入十六进制数
    cin>>dec>>x4; // 输入十进制数
    cout<<"x1="<<x1<<"\tx2="<<x2<<"\tx3="<<x3<<"\tx4="<<x4<<endl;
}
```


string 与字符串输入 / 输出

1 . string 类型

- string 是标准模板库 (STL) 中提供的字符串类型，可以像 int、char 等基本数据类型一样定义 string 类型的对象，以及用 ">、<、>=、<=、<>、=、+=" 等运算符进行各种字符运算。

2 . String 的定义及初始化

- string c; / 定义空字符串 c ,
- string c1("this is a string"); // 定义字符串 c1 , 用指定字符串初始化
- string c2=c1; // 定义字符串 c2 , 用 c1 初始化它
- string s[10]; // 定义能够保存 10 个字符串的数组，相当于 char[][];
- string s(5,'c'); // 定义 s , 用 5 个 ' c ' , 即 " ccccc " 初始化 ;

string 与字符串输入 / 输出

3 . string 类型的赋值

– string 类型的赋值操作与 int 等基本类型的赋值操作相同，不必用 strcpy 函数。例如，

```
string s1, s2,s3[3];           // 定义 string 对象及数组
```

```
//string 对象数组定义与初始化
```

```
string name[3] = { "tom","jerry","duck" };
```

```
s1 = "this is a string!";      //string 赋值
```

```
s2 = s1;
```

```
s3[0] = s1;                    //string 数组元素访问
```

```
s3[1]="string arr";
```

string 与字符串输入 / 输出

4 . string 类型的连接

– “+、 +=” 可以连接两个 string 类型对象，例如：

```
string s1("I am boy"), s3;
```

```
string s2 = "i come from china!";
```

```
s3 = s1 + ", " + s2;    //s3: I am boy , i come from china!
```

```
s1 += ", " + s2;        //s1: I am boy , i come from china!
```

string 与字符串输入 / 输出

• 4 . string 类型的输入输出和大小比较

- string 类型可以用 `cin` 和 `cout` 直接输入或输出；用 "`>`、`>=`、`==`、`<`、`<=`、`!=`、" 进行大小比较，比较的是两个 string 对象对应位置字符的 Ascii 码。

```
#include<iostream>
#include<string>
using namespace std;
int main(){
    string s1,s2,big;
    cout << " 输入两个字符串 :" << endl;
    cin >> s1 >> s2;
    cout << " 参加比较的两个字符串是 :" << s1 << "," << s2 << endl;
    if (s1 > s2) big = s1;
    else if (s1 == s2) big = "same";
    else big = s2;
    cout << " 大字符串是 :" << big << endl;
    return 0;
}
```

08



C++ 运算符



关键字是系统预定义的单词。C++ 不允许对关键字重定义。

C++ 常用的关键字：

auto break case char class const continue default delete else enum explicit extern float for friend
goto if inline int long new operator private protected public register return short signed sizeof
static struct switch this typedef union unsigned virtual void while

标识符：

语法：以字母或下划线开始，由字母、数字和下划线组成的符号串

- (1) 不能使用关键字作用户标识符；
- (2) C++ 中，字母大小写敏感；
- (3) C++ 没有规定标识符的长度，不同编译系统有不同的识别长度；
- (4) 标识符尽可能做到见文知义。





类型	说明	长度 (字节)	表示范围	备注
char	字符型	1	-128~127	$-2^7 \sim (2^7 - 1)$
int	整型	4	-2147483648~2147483647	$-2^{31} \sim (2^{31} - 1)$
double	双精度型	8	$-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$	15 位有效位

类型	说明	字节	范围
short [int]	短整型	2	-32768 ~ 32767
signed short [int]	有符号短整型	2	-32768 ~ 32767
unsigned short [int]	无符号短整型	2	0 ~ 65535
int	整型	4	-2147483648 ~ 2147483647
signed [int]	有符号整型	4	-2147483648 ~ 2147483647
unsigned [int]	无符号整型	4	0 ~ 4294967295
long [int]	长整型	4	-2147483648 ~ 2147483647
signed long [int]	有符号长整型	4	-2147483648 ~ 2147483647

C++ 运算符是指用于执行特定操作的符号或符号组合。以下是常见的 C++ 运算符：

1. 算术运算符：加（+）、减（-）、乘（*）、除（/）、取余（%）等。
2. 比较运算符：等于（==）、不等于（!=）、大于（>）、小于（<）、大于等于（>=）、小于等于（<=）等。
3. 逻辑运算符：与（&&）、或（||）、非（!）等。
4. 位运算符：按位与（&）、按位或（|）、异或（^）、左移（<<）、右移（>>）等。
5. 赋值运算符：赋值（=）、复合赋值（+=、-=、*=、/=、%=、&=、|=、^=、<<=、>>=）等。
6. 条件运算符：条件表达式（?:）。
7. 逗号运算符：逗号（,）。
8. 成员访问运算符：点（.）、箭头（->）。
9. 自增自减运算符：自增（++）、自减（--）。
10. 类型运算符：sizeof。
11. 强制类型转换运算符：static_cast、reinterpret_cast、dynamic_cast、const_cast。

算术运算符有：加（+）、减（-）、乘（*）、除（/）、取余（%）等。

（1）除（/）的取整：如果 / 的所有运算对象都是整数，/ 的功能就是取整。

$5/3 == 1;$

$5/2 == 2;$

（2）除（/）的除法运算符：如果 / 有一个运算对象是实型，/ 的功能就是除法运算。

$5/2.0 == 2.5;$

（3）取余（%）运算符。

$5\%2 == 1;$

$3\%5 == 3;$



```
#include <iostream>
using namespace std;
int main()
{
    int data = 0;
    cout << " 请输入一个整数： ";
    cin >> data;

    if (data % 3 == 0)
        cout << data << " 可以被 3 整除 " << endl;
    else
        cout << data << " 不可以被 3 整除 " << endl;

    return 0;
}
```

输出：
请输入一个整数： 16
16 不可以被 3 整除

- 使用 rand() 函数产生一个 60~100 的随机数。

// 60 = 60 + 0;

// 100 = 60 + 40;

// 因此, data%n=n-1; 推导出 n-1=40; 所以 n=41;

// 所以取余的数是 41。

rand()%41 + 60;

- 使用 rand() 函数产生一个 'a' ~ 'z' 的随机字母。

// 'a' = 'a' + 0;

// 'z' = 'a' + 25; (26 个字母, 从 0 开始算就是 0~25)

// 因此, data%n=n-1; 推导出 n-1=25; 所以 n=26;

// 所以取余的数是 41。

rand()%26 + 'a';

- `+=`、`-=`、`*=`、`/=`、`%=` 等。

```
a+=b;// a=a+b;
```

```
a-=b;// a=a-b;
```

```
a/=b;// a=a/b;
```

```
a%=b;// a=a%b;
```

- 特别需要注意的是：一定要把等号右边看成一个整体。

```
int a=3;
```

```
a*=4+5;//a=a*(4+5);
```

```
// a=27;
```

- 示例：

```
int a=0;
```

```
a+=a-=a*=a;// 根据运算符优先级，从右往左运算；
```

```
// a=0;
```

```
// 规律，只要看到中间有 a-=a；则必定为 0。
```

- 等于 (`==`)、不等于 (`!=`)、大于 (`>`)、小于 (`<`)、大于等于 (`>=`)、小于等于 (`<=`) 等。特别需要注意的是，等号一定要在右边。

`=<` 是错误的。

`=<` 是错误的。

- 示例：

- 与 (&&)、或 (||)、非 (!) 等。

(1) 与 (&&) : 只要有一个为假, 则整个表达式为假, 否则为真。

(2) 或 (||) : 只要有一个为真, 则整个表达式为真, 否则为假。

(3) 非 (!) : 把真变为假, 假变为真。

- 需要注意的是, && 和 || 都有一些特性:

➤ 如果 && 中检测到一个对象为假, 则直接判断为假, 因为后面的判断对结果已经不起影响了。

➤ 如果 || 中的检测到一个对象为真, 则直接判断为真, 因为后面的判断对结果已经不起影响了。

在 c 语言中, 除了 0 是假, 其他都为真。

按位与（&）、按位或（|）、异或（^）、左移（<<）、右移（>>）等。

● & 按位与

规律：全 1 为 1，有 0 为 0。

特点：和 1 相与保持不变，和 0 相与为 0。

场景：将指定位清零。

```
1100 0011
& 1111 0000
-----
1100 0000
```

示例：将一个字节的第三第四位清零，其他保持不变。

```
unsigned char data;
```

```
data=data & 0xe7;// 0xe7 = 1110 0111
```

```
data&=0xe7;// 等效
```

● | 按位或

规律：全 0 为 0，有 1 为 1。

特点：有 1 置 1，有 0 保持不变。

场景：将指定位置 1。

1100 0011

| 1111 0000

1111 0011

示例：将一个字节的第五、六个位置置 1，其他保持不变。

```
unsigned char data;
```

```
data=data | 0x60;// 0x60 = 0110 0000
```

```
data|=0x60;// 等效
```

● ~ 按位取反

规律：0 变 1，1 变 0。

一般用来配合其他位运算符。

$\sim 1100\ 0011 = 0011\ 1100$;

● ^ 按位异或

规律：相同为 0，不同为 1。

特点：和 1 异或取反，和 0 异或保持不变。

场景：将指定位发生翻转。

1100 0011

^ 1111 0000

0011 0011

位运算符（二进制位运算）

100

- << 左移：左边丢弃，右边补零

移动的位数不要超过自身位的宽度。

规律：左移多少，相当于乘上 2 的 n 次方。

位运算符（二进制位运算）

101

- << 左移：左边丢弃，右边补零

```
char data=0000 0001;
```

```
// 左移 0 位：
```

```
data=data<<0;    // = data*2^0;
```

```
// 左移 1 位：
```

```
data=data<<1;    // = data*2^1;
```

```
// 左移 2 位：
```

```
data=data<<2;    // = data*2^2;
```

```
// 左移 3 位：
```

```
data=data<<3;    // = data*2^3;
```

```
// .....
```

- **>> 右移：右边丢弃，左边补 0 或补 1**

算术右移、逻辑右移都是编译器决定的，用户无法确定。

无符号：右边丢弃，左边补 0。

有符号：

正数：右边丢弃，左边补 0。

负数：右边丢弃，左边补 1（算术左移）。

负数：右边丢弃，左边补 0（逻辑左移）。

规律：左移多少，相当于除上 2 的 n 次方。

>>> 位运算符（二进制位运算）

103

- >> 右移：右边丢弃，左边补 0 或补 1

```
unsigned char data=1000 0000;
```

```
// 左移 0 位：
```

```
data=data>>0;      // = data 除上  $2^0$ ;
```

```
// 左移 1 位：
```

```
data=data>>1;// = data 除上  $2^1$ ;
```

```
// 左移 2 位：
```

```
data=data>>2;// = data 除上  $2^2$ ;
```

```
// 左移 3 位：
```

```
data=data>>3;// = data 除上  $2^3$ ;
```

```
// .....
```



三目运算符 ? :

104

C++ 中的三目运算符是一种条件表达式，也称为条件运算符。它的语法如下：

`condition ? expression1 : expression2;`

其中，`condition` 是一个条件表达式，如果它的值为真，则返回 `expression1` 的值；否则，返回 `expression2` 的值。

例如，以下代码使用三目运算符计算两个数的最大值：

```
#include <iostream>
using namespace std;
int main() {
    int a = 10;
    int b = 20;
    int max_num = (a > b) ? a : b;
    cout << "Max number is " << max_num << endl;
    return 0;
}
```

输出结果为：Max number is 20

在上面的代码中，如果 `a` 大于 `b`，则 `max_num` 等于 `a`，否则 `max_num` 等于 `b`。

>>> 运算符优先级

C++ 中运算符的优先级如下（从高到低）：

注意，优先级从上到下，越靠上的优先级越高。例如，括号优先级最高，因此括号内的运算会先被执行。而逗号运算符的优先级最低，因此它的运算会在其他运算完成后才会被执行。

优先级	运算符类型	运算符
1	括号	()
2	一元运算符	++ -- ! ~ + - * & sizeof new delete
3	强制类型转换	(type)
4	乘除模运算符	* / %
5	加减运算符	+ -
6	移位运算符	<< >>
7	关系运算符	< <= > >=
8	相等运算符	== !=
9	按位与运算符	&
10	按位异或运算符	^
11	按位或运算符	
12	逻辑与运算符	&&
13	逻辑或运算符	
14	条件运算符	?:
15	赋值运算符	= += -= *= /= %= <<= >>= &= ^= =
16	逗号运算符	,

C++ 中的自增运算符 (`++`) 和自减运算符 (`--`) 是一种特殊的算术运算符，用于增加或减少一个变量的值。它们可以作为前缀或后缀操作符使用。

前缀形式：操作符位于变量名之前（例如 `++i`），此时变量的值会在计算前先自增或自减一次，然后返回该变量的新值。

后缀形式：操作符位于变量名之后（例如 `i++`），此时变量的值会在计算之后自增或自减一次，但是表达式的结果仍为原始值，只有下一次使用该变量时才会体现出自增或自减的效果。

示例：



```
int i = 1;  
cout << ++i; // 输出 2 , 因为 i 先自增一次再输出  
cout << i++; // 输出 2 , 但 i 的值已经自增了一次 , 下一次使用 i 时就会体现出来  
cout << i; // 输出 3 , 因为 i 自增了一次
```



THANKS!